TP : Le Monde du Wumpus

Description du problème

Vous devez écrire le programme d'un agent capable de jouer au jeu du Wumpus. La description **PEAS** de la tâche est la suivante :

— **P**erformance :
  — Tomber dans un puits ou être mangé par un Wumpus : $-\infty$
  — Déplacement vers une case de coût a : $-a$
  — But : ramasser tout les lingots
— **E**nvironnement : grille carrée $n \times n$, plusieurs lingots d'or, plusieurs puits, plusieurs Wumpus. On note (i, j) la case de la ligne i et de la colonne j. La case $(0, 0)$ en haut à gauche est la position initial de l'agent.
— **A**ctions / Effecteurs : `'h'` : aller en haut, `'d'` : aller à droite, `'b'` : aller en bas, `'g'` : aller à gauche, `'s'` : saisir le lingot de la case courante, `'q'` : quitter la partie.
— **S**enseurs : la carte est entièrement connue à l'avance (agent omniscient)

1 Planification optimale par A*

1.1 Énoncé

Étant donnée une carte **entièrement visible**, calculez la **meilleure séquence d'actions** : partir de la case de départ, ramasser **TOUT** les lingots et quitter. Votre programme écrit cette séquence sur la sortie standard, où elle sera vérifiée automatiquement. S'il n'est pas possible de ramasser tous les lingots, saisissez simplement la solution `'q'`.

1.2 Format de la carte

La carte est décrite dans un fichier texte de n lignes de n caractères :

Caractère	Signification
`1` à `9`	case vide : le chiffre est le coût d'*entrée* dans la case
`$`	lingot d'or : coût d'entrée 1
`P`	puits : mortel, donc infranchissable
`W`	Wumpus : mortel, donc infranchissable

Sortir de la grille est interdit. **Le coût d'un déplacement vaut toujours au moins 1** : il n'existe pas de case de coût nul.

Exemple

```
115$
1P91
$3P1
1511
```

Départ en $(0, 0)$, lingots en $(2, 0)$ et $(0, 3)$.
La séquence optimale est `"bbshhdddsq"`, de coût 11.

1.3 Interface de votre programme

Votre programme est appelé avec le chemin de la carte en premier argument :

```
$ ./agent carte.txt
bbshhdddsq
```

— Il écrit sur la **sortie standard une seule ligne** : la séquence d'actions, suivie d'un saut de ligne. **Rien d'autre ne doit être écrit sur** `stdout` (vos messages debug vont sur `stderr`).

— La séquence est jouée depuis la case (0, 0) ; elle doit contenir une action `'s'` sur chaque case contenant un lingot et se terminer par `'q'`.

— Si plusieurs séquences optimales existent, n'importe laquelle est acceptée : c'est le **coût total** qui est comparé à l'optimum par le vérificateur.

— Toutes les cartes de test admettent une solution.

1.4 Indications

— Un état n'est pas réductible à une position : il faut mémoriser les lingots déjà ramassés.

— L'heuristique doit être **admissible** (ne jamais surestimer le coût restant), sinon la séquence renvoyée peut ne pas être optimale.

1.5 Barème

Critère	Points
Le programme renvoie toujours une séquence valide (actions légales, aucune case mortelle, tous les lingots ramassés, sortie correcte)	2.5
La séquence renvoyée est toujours de coût minimal	2.5
Efficacité du programme (algorithme + structure de données + heuristique)	5.0